Surgical Workflow Analysis

Medical Robotics Project

Christian Corvino
Student ID: 2268254
corvino.2268254@studenti.uniroma1.it

Giorgia Chiaro
Student ID: 2055497
chiaro.2055497@studenti.uniroma1.it

Marta Cudia
Student ID: 2047806
cudia.2047806@studenti.uniroma1.it

Alessio Olivieri
Student ID: 1973323
olivieri.1973323@studenti.uniroma1.it

July 19, 2026

Contents

1 Introduction **4**
- 1.1 Clinical Context and Surgical Workflow Analysis . . . 4
- 1.2 The Phase Recognition Problem . . . 4
- 1.3 Project Objective . . . 4
- 1.4 Use of Generative AI . . . 4

2 Dataset Analysis and Preparation **5**
- 2.1 Preliminary Data Verification . . . 5
- 2.2 Frame Extraction . . . 6
- 2.3 Frame-Label Alignment . . . 6
- 2.4 Train / Validation / Test Split . . . 6
- 2.5 Class Imbalance . . . 7

3 Methodology and Architectures **8**
- 3.1 Spatial Feature Extraction . . . 8
  - 3.1.1 Pipelines 1 and 2: Direct ImageNet Features (`teammates`, `original`) . . . 8
  - 3.1.2 Pipeline 3: Features from Fine-Tuned ResNet50 (`finetuned`) . . . 8
  - 3.1.3 Motivation for Comparison . . . 9
- 3.2 Compared Temporal Architectures . . . 9
  - 3.2.1 Temporal Transformer . . . 9
  - 3.2.2 Multi-Stage Temporal Convolutional Network (MS-TCN) . . . 9
  - 3.2.3 Multi-Task Extension: Surgical Instrument Detection . . . 10
  - 3.2.4 Recurrent Baselines: GRU, BiGRU, and MS-CBiGRU . . . 10
  - 3.2.5 SF-TMN with ASFormer Backbone . . . 10
- 3.3 Class Imbalance Management . . . 11
  - 3.3.1 Class Weights and Balanced Sampling . . . 11
  - 3.3.2 Compared Loss Function Families . . . 11
- 3.4 Training Pipeline . . . 12
  - 3.4.1 Data Preparation and Augmentation . . . 12
  - 3.4.2 Optimization . . . 12
  - 3.4.3 Model Selection and Early Stopping Criterion . . . 12

4 Post Processing and Metrics **13**
- 4.1 Post-Processing with Viterbi . . . 13
  - 4.1.1 Transition Model Estimation . . . 13
  - 4.1.2 Viterbi Decoding . . . 13
  - 4.1.3 Selection Between Raw and Post-Processed Prediction . . . 13
- 4.2 Frame-Level Evaluation Metrics . . . 14
- 4.3 Temporal Segmentation Metrics . . . 14
  - 4.3.1 Normalized Edit Score . . . 14
  - 4.3.2 F1@k: Segment-Level F1-score . . . 15

4.4 Multi-Label Metrics for Instrument Recognition 15
4.5 Computational Efficiency Metrics 15
4.6 Evaluation Protocol and Data Leakage Prevention 15

5 Results **17**
5.1 Comparison Configuration 17
5.2 Phase Recognition: Frame-Level Metrics 17
5.3 Temporal Quality of Predictions 18
5.4 Effect of Features and Depth 19
5.5 Auxiliary Task: Instruments 19
5.6 Single-Task MS-TCN Ablation 20
5.7 Computational Efficiency 20

6 Conclusions and Future Developments **21**
6.1 Study Limitations 21
6.2 Future Developments 22

List of Tables

2.1 Distribution of videos and frames across the training, validation, and test sets. . 7

3.1 Main hyperparameters of the compared architectures. 12

5.1 Frame-level metrics on the test set. M = macro; values in percentage. 17
5.2 Temporal segmentation metrics on the test set; values in percentage. 18
5.3 Multi-label instrument metrics on the test set; values in percentage. The threshold was selected exclusively on the validation set. 19
5.4 Preliminary ablation of the 10-layer single-task MS-TCN with fine-tuned features; values in percentage. 20
5.5 Computational cost observed on the three feature sources. The intervals reflect differences in the number of epochs until early stopping and in the actual duration of the executions. 20

Chapter 1

Introduction

1.1 Clinical Context and Surgical Workflow Analysis

Laparoscopic cholecystectomy is one of the most frequently performed minimally invasive surgical procedures worldwide, aiming at the removal of the gallbladder. With the advent of the digital operating room, surgical workflow analysis has gained fundamental importance [1]. Automatically understanding and modeling the different phases of a procedure from sensory data allows for optimizing hospital resource management, improving surgical resident training, and developing intelligent support systems to provide context-aware assistance directly in the operating room [2].

1.2 The Phase Recognition Problem

Despite the wide availability of videos recorded daily via endoscopes, the automatic recognition of the current surgical phase in real-time represents a significant computational challenge. Laparoscopic video streams are characterized by high visual variability due to factors such as uneven lighting, the presence of smoke or blood, visual artifacts, and instrument occlusion. Added to this is the heterogeneity of the surgical technique, which varies from surgeon to surgeon. Furthermore, transitions between phases can be ambiguous and difficult to delineate by analyzing a single frame out of its temporal context [3].

1.3 Project Objective

The main objective of this project is the development of a Deep Learning model for the automatic and robust recognition of the current surgical phase from a laparoscopic video stream. Using the reference dataset Cholec80, the project addresses three main tasks: (1) temporal modeling of video sequences to ensure logical consistency of predictions, (2) managing the intrinsic class imbalance, given that some phases of the intervention have a significantly longer duration compared to others, and (3) evaluating performance using workflow-specific metrics, going beyond simple overall accuracy.

1.4 Use of Generative AI

Generative Artificial Intelligence tools were used as a support tool during the preparation of this report, primarily for language refinement, structural organization, and clarification of technical descriptions. All methodological choices, implemented solutions, and analytical interpretations are the result of the author's work.

Chapter 2

Dataset Analysis and Preparation

The reference dataset for the project is Cholec80, published by the CAMMA research group at the University of Strasbourg, which includes 80 laparoscopic cholecystectomy videos recorded at 25 fps, each annotated frame by frame with one of 7 possible surgical phases: *Preparation*, *Calot Triangle Dissection*, *Clipping and Cutting*, *Gallbladder Dissection*, *Gallbladder Packaging*, *Cleaning and Coagulation*, *Gallbladder Retraction*. This chapter describes the preprocessing pipeline developed to transform the raw videos and their annotations into a structured, verified dataset ready for model training, justifying the design choices made, the correctness checks performed, and the obtained results.

2.1 Preliminary Data Verification

The dataset provides, for each video, a *phase annotations* file (`Frame`, `Phase` columns — one label for each frame at 25 fps, 0-based index, all frames annotated without downsampling) and a *tool annotations* file, generated instead at 1 fps and not directly used in this phase, but relevant as a reference for choosing the sampling frequency adopted during extraction (Section 2.2).

Before building any automated pipeline, the format of the phase files was manually inspected on a sample video (`video01`), to verify that the structure declared in the README actually matched the data. The check confirmed that the file contains 43,326 rows, consistent with a 25 fps video of a few dozen minutes, with continuous indexing and no skips; that the phases are encoded as strings in CamelCase format without spaces (e.g., `CalotTriangleDissection`), an important detail to avoid subsequent mapping errors; and that exactly the 7 phases expected from the README are present.

A systematic check on the phases actually present in each of the 80 annotation files, conducted to identify any anomalies before they could propagate undetected in subsequent phases, revealed that not all videos contain all 7 phases: 65 videos are "complete", 9 lack the *Preparation* phase, and 6 lack the *Cleaning and Coagulation* phase.

To determine whether this was an annotation error or a real characteristic of the data, a precise check was performed on the beginning and end of the affected files: for videos lacking *Preparation*, the first row is already labeled with the subsequent phase (*Calot Triangle Dissection*), not with an anomalous label; for videos lacking *Cleaning and Coagulation*, the last row coincides with the end of the *Gallbladder Retraction* phase, without interruptions halfway through another phase. The pattern is therefore consistent with recordings that do not cover the entire duration of the intervention (beginning or end not recorded), a real characteristic of a minor subset of acquisitions and not a parsing error. This information was used later to build a stratified split (Section 2.4).

2.2 Frame Extraction

The Cholec80 videos are recorded at 25 fps: an intervention of average duration (30–60 minutes) corresponds to tens of thousands of frames. Using all the original frames presents two practical problems: computational overhead, since consecutive frames at a distance of 1/25 of a second are visually almost identical and do not add useful information to distinguish the surgical phase; and a practical storage problem, since the work environment (Kaggle Notebooks) imposes a disk space limit of approximately 20 GB, which would be largely exceeded by saving all frames at full resolution for 80 videos.

A downsampling was applied that reduces the frequency from 25 fps to 1 fps, i.e., one frame for every 25 original ones. The choice is motivated by the fact that surgical phases have a typical duration on the order of minutes, so a 1-second sampling captures their evolution without loss of relevant information; furthermore, this frequency is consistent with that used by the dataset itself for tool annotations, aligning the preprocessing with the conventions of the literature on Cholec80.

In addition to temporal downsampling, each extracted frame was resized to 224×224 pixels and saved in JPEG format with a quality of 85. This choice is motivated by three factors: the architectures used as feature extractors require fixed-size inputs, typically 224×224; resizing during extraction, rather than a posteriori, drastically reduces the occupied disk space; a moderate JPEG compression offers a good compromise between visual quality and file size.

The procedure was successfully applied to all 80 videos, without errors, and with a resulting sampling factor equal to 25 for every single video — confirming that all videos share the same original frame rate of 25 fps. The number of extracted frames per video varies from a minimum of 740 (`video78`) to a maximum of 5,994 (`video58`), reflecting the natural variability in the duration of real surgical interventions (from approximately 12 to approximately 100 minutes).

2.3 Frame-Label Alignment

Since the extracted frames are at 1 fps while the official annotations are at 25 fps, each saved frame (index `saved_idx`) was associated with the row `original_frame_idx = saved_idx` $\times$ `25` of the original phase file, mapping the textual phase to an integer (0–6) via a dictionary built on the 7 official phases. The result is a single table (`full_dataset.csv`) with image path, source video, and label.

Verification: the total number of generated rows (184,578) coincides exactly with the sum of the frames extracted during the extraction phase, and all 80 videos are represented — no loss or duplication. This table constitutes the standard interface between preprocessing and the subsequent phases of the project (Dataset/DataLoader construction and model training).

2.4 Train / Validation / Test Split

A random split at the single-frame level would be incorrect: consecutive frames from the same video are almost identical — same anatomy, same patient, same lighting, same instruments in the scene — so mixing them between training and testing would produce *data leakage*, i.e., an artificial overestimation of the model's performance. The model would seem to perform well in validation and testing only because it has already seen almost identical frames during training, not because it has truly learned to generalize to unseen patients and recordings.

The split was therefore done at the whole-video level, assigning the entire video (and all its frames) to a single set. A **sequential** split based on the video identifier was adopted, in the proportion of **32 / 8 / 40** (training / validation / test), also used in the literature for the Cholec80 dataset [4]:

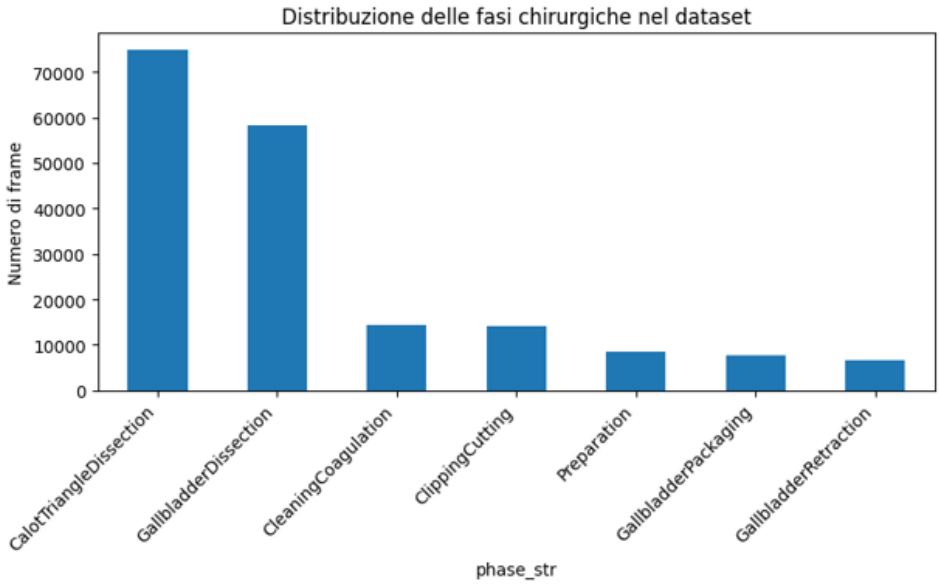
Set	Videos	Frames
Training	32	71,032
Validation	8	15,312
Test	40	98,234

Table 2.1: Distribution of videos and frames across the training, validation, and test sets.

It was explicitly verified that no video appears in more than one set (empty intersection between the identifiers of the three groups) and that all 7 phases are represented in each set.

2.5 Class Imbalance

The distribution of the 184,578 frames per phase reflects the true relative duration of each surgical phase, which is highly imbalanced: the two main phases (*Calot Triangle Dissection* and *Gallbladder Dissection*) alone cover over 72% of the dataset, while the rarest phase (*Gallbladder Retraction*, 3.6%) is represented about 11 times less than the most frequent one.



This imbalance is consistent with the clinical nature of the intervention and is therefore not a preprocessing artifact, but an intrinsic characteristic of the task, which must be explicitly managed during the training phase to prevent the model from systematically favoring the majority classes at the expense of the minority ones. As a basis for this management, class weights inversely proportional to the frequency in the training set (`class_weights.csv`) were calculated, to be provided to the model's loss function, thus penalizing more heavily the errors made on rare classes.

Chapter 3

Methodology and Architectures

3.1 Spatial Feature Extraction

Following the literature, each video frame is reduced to a feature vector via a pre-trained CNN, delegating the modeling of inter-frame dependencies to the temporal component (Chapter 4). **Three extraction pipelines** were produced and compared, all based on ResNet50 but differing in pre-processing and backbone training. All of them produce a 2048-feature vector per frame (output of the ResNet50 global average pooling, after removing the classification layer), sampled at 1 FPS.

3.1.1 Pipelines 1 and 2: Direct ImageNet Features (`teammates`, `original`)

Both pipelines use a ResNet50 pre-trained on ImageNet (`IMAGENET1K_V2` weights) as an "off-the-shelf" extractor, with a frozen backbone and standard ImageNet normalization. They differ only in their implementation: the first one (`teammates`) physically extracts the frames at 1 FPS, resizes them to 224×224, and saves them to disk as JPEGs before inference; the second one (`original`) decodes the video in a single pass, reading only the target frames (`cap.read()`) and discarding the others without full decoding (`cap.grab()`), avoiding intermediate I/O to disk and including an automatic CPU fallback in case of GPU error.

3.1.2 Pipeline 3: Features from Fine-Tuned ResNet50 (`finetuned`)

The third pipeline performs a **supervised fine-tuning** of the backbone on the phase classification task, under the hypothesis that specialized features on the laparoscopic domain are more informative than generic ImageNet features.

The annotations (originally at 25 FPS) are downsampled to 1 FPS, and the corresponding frames are pre-extracted into a shared cache (256×256). The fine-tuning takes place on a reduced subset of 32 videos (27 training / 5 validation, from the original training split), unfreezing only the last two residual blocks (`layer3`, `layer4`) and the final classifier (Dropout 0.3 + 7-class Linear layer), with Batch Normalization statistics frozen in the un-trained layers. The optimization uses AdamW with differentiated learning rates per group (1×10^{-4} on convolutional layers, 1×10^{-3} on the classifier), a weight decay of 1×10^{-4}, Cross-Entropy with 0.1 label smoothing, `WeightedRandomSampler` for class imbalance, and data augmentation (random crop, flip, color jitter) only during training. Training proceeds for a maximum of 15 epochs with early stopping (*patience* 4) on the validation macro F1 score.

At the end, the classifier is replaced with an identity layer, reverting the network back to a pure feature extractor — but specialized for the domain — which is applied to **all 80 videos** of the dataset to produce the `finetuned` set used in the subsequent chapters.

3.1.3 Motivation for Comparison

Two conceptually equivalent but implementation-wise distinct pipelines (`teammates`, `original`) and a third one with supervised adaptation of the backbone (`finetuned`) allow to experimentally isolate the effect of spatial feature quality on recognition performance, independently of the chosen temporal modeling architecture. The quantitative comparison is resumed in Chapter 6.

3.2 Compared Temporal Architectures

To model the temporal dependencies between frames, four families of architectures were compared: recurrent models (GRU and BiGRU), a window-based temporal Transformer, multi-stage temporal convolutional networks (MS-TCN, also in a multi-task variant), and SF-TMN with ASFormer blocks. All models receive the same 2048-dimensional feature sequences as input and output a phase prediction for each second of the video. In this way, the comparison focuses on the temporal module and the feature source, keeping the seven-phase recognition task fixed.

3.2.1 Temporal Transformer

The ResNet50 features are first projected into a lower-dimensional embedding space through a *linear projector* (`LinearProjector`), consisting of two sequential Linear–ReLU–Dropout blocks ($2048 \to 256 \to 256$, dropout 0.2), which reduces the dimensionality and introduces non-linearity prior to entering the temporal module.

Since the self-attention mechanism is structurally invariant to sequence order, position information is added via classical sinusoidal *positional encoding* (Vaswani et al.), which is added to the projected embedding. The resulting sequence is processed by a `TransformerEncoder` with 3 layers, 8 attention heads, model dimension $d_{model} = 256$, internal feed-forward dimension 512, and dropout 0.2. The output, normalized via LayerNorm, is finally projected onto the 7 phase classes through a linear layer.

The model operates on fixed-length temporal windows of $W = 64$ frames: during training, windows are extracted with a stride of 1 (maximum overlap, very dense dataset), while during inference a stride of 32 frames is used, with the predictions of the overlapping windows aggregated by averaging the logits prior to the final softmax. This overlapping window inference scheme allows to cover videos of arbitrary length while maintaining a limited and computationally manageable temporal context for each single inference.

3.2.2 Multi-Stage Temporal Convolutional Network (MS-TCN)

Unlike the Transformer, the MS-TCN processes the entire video sequence in a single pass, without windowing.

The base block is the `DilatedResidualLayer`: a dilated 1D convolution (kernel 3, dilation 2^i at layer i) followed by a 1×1 convolution and a residual connection, with 0.5 dropout. Stacking 11 layers of this type with increasing dilation $(2^0, 2^1, \ldots, 2^{10})$ yields an exponentially large receptive field while keeping the network relatively lightweight (64 channels per layer). A single stage constructed in this way (`SingleStageTCN`) maps the input features into a preliminary phase prediction.

The distinctive feature of the MS-TCN is the *multi-stage* structure: the output of the first stage (after softmax) is passed again as input to a subsequent stage identical in structure, and so on for a total of 4 cascaded stages. Each subsequent stage thus receives as input not the original features but the probability distribution predicted by the previous stage, achieving an iterative refinement of the predictions that helps correct local errors and reduce the fragmentation of the predicted phases.

Implementation Note. Unlike the original formulation of TeCNO, which uses *causal* convolutions to allow truly online inference, the implementation adopted here uses dilated convolutions with symmetric padding (non-causal): the model therefore has access, for each frame, also to a future context within the same sequence. This choice implies an offline operation (the entire video must be available before inference) but generally improves accuracy compared to the causal variant, being an acceptable compromise for a retrospective workflow analysis project.

3.2.3 Multi-Task Extension: Surgical Instrument Detection

As a variant of the MS-TCN, a multi-task version (`MultiTaskMSGCN`) was also implemented, which shares the same convolutional backbone but couples the main phase recognition task with a second auxiliary task of detecting the presence of surgical instruments (7 instruments, binary multi-label classification: Grasper, Bipolar, Hook, Scissors, Clipper, Irrigator, SpecimenBag). Each stage thus produces two parallel outputs via two distinct convolutional heads (`conv_out_phases` and `conv_out_tools`), while sharing all internal convolutional layers. This approach follows the logic of multi-task learning: the additional supervision on the presence of instruments provides a correlated but distinct signal with respect to the phase, which can act as a regularizer and help the model learn more informative representations, particularly for the less represented phases in the dataset.

To verify the effect of depth, two configurations of the multi-task variant were trained, both with 4 stages and 64 feature maps but with either 10 or 11 dilated layers per stage. The two models have 796,792 and 862,840 parameters respectively. The output folder informally named `10heads` therefore contains the model with *10 layers per stage*; the term "heads" does not describe the structure of the MS-TCN and is not used in the following.

3.2.4 Recurrent Baselines: GRU, BiGRU, and MS-CBiGRU

Three recurrent models were included as compact baselines. In all cases, the features are projected from 2048 to 256 dimensions using a linear layer followed by LayerNorm, GELU, and a 0.30 dropout. The **GRU** uses two unidirectional recurrent layers with a hidden state size of 256; the **BiGRU** uses two bidirectional layers with 128 units per direction, maintaining an overall output of 256 dimensions. Both end with dropout and a linear classifier on the seven phases.

The **MS-CBiGRU** variant adds, before the BiGRU, a multi-scale temporal convolutional block with four parallel branches of kernel 3 and dilations $1, 2, 4, 8$. The outputs of the branches are concatenated, fused via 1×1 convolution, and added to the input with a residual connection and LayerNorm. The block provides the BiGRU with local descriptors at different temporal scales, thus combining convolutional context and bidirectional memory. The three models have approximately 1.316, 1.120, and 1.383 million parameters, respectively.

3.2.5 SF-TMN with ASFormer Backbone

The **SF-TMN (ASFormer)** model processes the sequence in parallel at two temporal resolutions. The *slow* pathway operates at the original resolution of 1 fps, while the *fast* pathway applies max-pooling on segments of 32 frames. Each pathway uses an ASFormer stage with 10 layers, 64 channels, dilated convolutions, and sliding-window local self-attention. The slow and fast features are realigned and fused using learned scalar weights normalized with softmax.

The fused prediction is refined by three pairs of ASFormer decoders; at each level, the fast pathway is updated on its own scale, while the slow pathway receives the combined prediction from the previous level. All intermediate outputs contribute to the loss. The final configuration has 2,259,932 parameters, uses 32-second segments, and employs gradient checkpointing to constrain memory. Unlike the Transformer and MS-TCN candidates, this implementation follows the reference configuration with unweighted cross-entropy and truncated smoothing loss.

3.3 Class Imbalance Management

The dataset presents an intrinsic imbalance among the 7 surgical phases, some of which (e.g., *CalotTriangleDissection*, *GallbladderDissection*) occupy a considerably larger portion of the videos compared to others (e.g., *ClippingCutting*, *GallbladderRetraction*). Several strategies were adopted, both at the data sampling level and at the loss function level.

3.3.1 Class Weights and Balanced Sampling

For models using a weighted loss, a class weight vector is calculated, which is inversely proportional to the frequency of the class itself in the training set:

$$w_c = \frac{N}{K \cdot n_c}$$

where N is the total number of training frames, $K = 7$ is the number of classes, and n_c is the number of frames of class c. These weights are used as the `weight` argument in cross-entropy based losses. The Transformer also employs a `WeightedRandomSampler` that assigns higher probability to windows centered on rare classes. MS-TCN and recurrent models, processing the entire video as a single sequence, do not perform window-level sampling. SF-TMN is a deliberate exception: to respect its reference configuration, it uses standard cross-entropy without class weights.

3.3.2 Compared Loss Function Families

For the Transformer and MS-TCN, three families of losses for the phase classification task were implemented and experimentally compared:

- **Weighted Cross-Entropy**: standard cross-entropy weighted by class, taken as a baseline.
- **Focal Loss**:
  $$\mathcal{L}_{focal} = -(1 - p_t)^{\gamma} \log(p_t)$$
  which reduces the loss contribution of already well-classified samples (high confidence), focusing the gradient on difficult or rare cases. A $\gamma = 2$ was used for the Transformer and $\gamma = 1$ for the MS-TCN, both variants with 0.1 label smoothing.
- **Boundary-Aware Loss**: a custom loss that assigns higher weight to frames near phase transitions (within a radius of 5 frames, with a triangular weight kernel up to a factor of 3×), based on the observation that classification errors tend to concentrate exactly at phase changes.

For the MS-TCN, a **truncated smoothing loss** is added to each of these losses, applied to each stage of the network:

$$\mathcal{L}_{smooth} = \frac{1}{TK} \sum_{t,c} \min\left(\left(\log \hat{y}_{t,c} - \log \hat{y}_{t-1,c} \right)^2, \tau \right)$$

with $\tau = 16$, weighted with a coefficient of 0.30 in the total loss. This term, taken from the original MS-TCN formulation, penalizes abrupt variations between the log-probabilities of temporally adjacent frames, encouraging more coherent and less fragmented predictions.

In the multi-task variant, the total loss for each stage also sums a **Binary Cross-Entropy** term for the instrument detection task, weighted with a coefficient of 0.40:

$$\mathcal{L}_{tot} = \sum_{s=1}^{4} \left(\mathcal{L}_{phase}^{(s)} + 0.30\, \mathcal{L}_{smooth}^{(s)} + 0.40\, \mathcal{L}_{tools}^{(s)} \right)$$

The use of a balanced positive weight for the BCE (`pos_weight`) is supported but disabled in the current configuration (`USE_TOOL_POSITIVE_WEIGHTS = False`).

For GRU, BiGRU, and MS-CBiGRU, the loss consists of weighted cross-entropy with 0.1 label smoothing plus truncated temporal smoothing with a 0.15 coefficient. SF-TMN instead uses unweighted cross-entropy and temporal smoothing with a 0.15 coefficient, summed over the four outputs (fused encoder and three refinements). These choices are kept fixed across the three feature sources.

3.4 Training Pipeline

3.4.1 Data Preparation and Augmentation

The final experiments adopt a fixed split common to all models: videos 1–32 for training, 33–40 for validation, and 41–80 for testing, with seed 42. During the training of Transformer and MS-TCN, a mild augmentation is applied to the features: additive Gaussian noise ($\sigma = 0.01$) and, with a 0.3 probability per batch, random feature dropout (zeroing out 5% of the components). For the Transformer only, a *speed perturbation* between 0.9× and 1.1× is added. The recurrent baselines and SF-TMN do not apply these augmentations, in accordance with the final notebooks.

3.4.2 Optimization

Transformer, MS-TCN, and recurrent models use **AdamW**; SF-TMN uses **Adam**, consistent with the reference implementation. The Transformer adopts `CosineAnnealingWarmRestarts`, the MS-TCN a linear warmup of 5 epochs followed by `CosineAnnealingLR`, while recurrent models and SF-TMN use `ReduceLROnPlateau`. All experiments use mixed precision and gradient clipping. The main hyperparameters are summarized in Table 3.1.

Model	Optim.	LR	Weight decay	Epochs	Patience
GRU / BiGRU / MS-CBiGRU	AdamW	3×10^{-4}	1×10^{-5}	80	12
Transformer	AdamW	2×10^{-4}	1×10^{-2}	40	5
MS-TCN / MS-TCN multi-task	AdamW	5×10^{-4}	1×10^{-5}	150	20
SF-TMN(ASFormer)	Adam	1×10^{-4}	1×10^{-5}	200	25

Table 3.1: Main hyperparameters of the compared architectures.

3.4.3 Model Selection and Early Stopping Criterion

For all architectures, the training adopts early stopping based on the primary validation metric, the **mean segmental F1** (average of F1@10, F1@25, and F1@50, defined in Chapter 5), rather than just frame-by-frame accuracy. The checkpoint with the best primary metric on the validation set is saved and reloaded for the final evaluation. For Transformer and MS-TCN, validation also selects the candidate loss; for each model, it additionally chooses between raw and Viterbi predictions. The entire pipeline was executed on GPU in a Kaggle environment, tracking training time, VRAM memory, and inference time.

Chapter 4

Post Processing and Metrics

4.1 Post-Processing with Viterbi

The frame-by-frame predictions produced by the models described in Chapter 4, although accurate, tend to present a certain degree of fragmentation: brief oscillations between phases even in temporally stable video portions, due to local noise in visual features or momentary ambiguities. Since a cholecystectomy follows a largely predictable phase order, this structure can be exploited to correct the most isolated errors.

For this purpose, an approach based on an implicit **Hidden Markov Model** was adopted, in which the model's raw predictions are treated as observed emissions and the true phase sequence is estimated a posteriori using the **Viterbi** algorithm.

4.1.1 Transition Model Estimation

A phase transition model is empirically estimated from the training set: for each pair of consecutive phases (c_i, c_j) observed in the training sequences, the frequency of the transition $c_i \to c_j$ is counted, yielding a transition matrix $A \in \mathbb{R}^{7\times 7}$ (with additive smoothing $\varepsilon = 10^{-3}$ to avoid zero probabilities on unobserved transitions) and an initial probability vector π on the opening phase of the video. Both are normalized by row (in the case of A) or globally (in the case of π) and converted into log-probabilities.

4.1.2 Viterbi Decoding

Once inference is complete, for each video the model provides a sequence of log-probabilities $\log p(y_t \mid x_t)$ for each frame t. The Viterbi algorithm identifies the phase sequence $\hat{y}_1, \ldots, \hat{y}_T$ that jointly maximizes the likelihood of the emissions and the coherence of the transitions:

$$\hat{y}_{1:T} = \arg\max_{y_{1:T}} \left[\log \pi_{y_1} + \log p(y_1 \mid x_1) + \sum_{t=2}^{T} \Big(\log A_{y_{t-1}, y_t} + \log p(y_t \mid x_t) \Big) \right]$$

The search for the optimal path is performed via dynamic programming, with a complexity of $O(T \cdot K^2)$ ($K = 7$ phases), making the operation computationally negligible even for long videos. The result is a phase sequence that respects the transition structure learned from real data, implicitly penalizing prediction sequences that are inconsistent with the normal course of the intervention (e.g., an implausible return to a previous phase).

4.1.3 Selection Between Raw and Post-Processed Prediction

For each combination of feature source, architecture, and loss function, both the raw prediction (*raw*) and the post-processed one with Viterbi are evaluated on the validation set, and the variant

with the highest mean segmental F1 metric (using Edit Score and macro F1 as disambiguation criteria) is selected. This ensures that post-processing is adopted only when it effectively improves prediction quality, avoiding the introduction of a systematic bias towards excessively "smooth" sequences at the expense of sensitivity to real phase transitions poorly detected by the model.

4.2 Frame-Level Evaluation Metrics

Global accuracy alone (percentage of correctly classified frames) is an insufficient metric in the presence of strong class imbalance, as it can be dominated by the most frequent phases. Therefore, for each phase c, Precision, Recall, F1-score, and Jaccard Index (Intersection over Union at the frame level) are calculated, and subsequently aggregated using the macro average (unweighted arithmetic mean over the 7 classes), in order to give equal importance to each phase regardless of its duration in the dataset:

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}, \qquad \text{Recall}_c = \frac{TP_c}{TP_c + FN_c},$$
$$\text{F1}_c = \frac{2\,\text{Precision}_c\,\text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}, \quad \text{Jaccard}_c = \frac{TP_c}{TP_c + FP_c + FN_c}.$$

The **macro** metrics are the unweighted average of the per-class values and constitute the main reference, because they do not hide errors on rare phases. The **weighted** aggregations, present in the preliminary ablation, instead weight each class by its support and are thus closer to accuracy; the **micro** metrics directly aggregate TP, FP, and FN before calculating the ratio. In addition to the pooled aggregation of all frames, the per-video mean and standard deviation of accuracy, macro precision, macro recall, and macro Jaccard are saved, which are useful for measuring variability across interventions. The per-class report and the confusion matrix complete the diagnostic analysis by showing which phases are confused most frequently.

4.3 Temporal Segmentation Metrics

Frame-level metrics, while necessary, do not adequately capture the prediction quality from the perspective of the workflow's temporal structure: a model can achieve high frame-level accuracy while producing an excessive number of short spurious segments (phases that "flicker" for a few frames), a behavior that is unrealistic from a clinical standpoint and of little use for a practical context-aware assistance application.

4.3.1 Normalized Edit Score

The prediction and ground truth sequences are first reduced to sequences of segments, grouping consecutive frames with the same label (`extract_segments` function). The Levenshtein distance is then calculated between the label sequence of the predicted segments and that of the real segments, normalized with respect to the maximum number of segments between the two sequences:

$$\text{Edit} = 1 - \frac{\text{LevenshteinDistance}(\text{segments}_{\text{pred}}, \text{segments}_{\text{gt}})}{\max(|\text{segments}_{\text{pred}}|, |\text{segments}_{\text{gt}}|)}$$

This metric, evaluated at the segment sequence level rather than the single-frame level, is particularly sensitive to fragmentation: a model that introduces numerous spurious segments produces a segment sequence much longer than the real one, heavily penalizing the Edit Score even with a relatively high frame-level accuracy.

4.3.2 F1@k: Segment-Level F1-score

To evaluate the quality of predictions at the single phase segment level, the F1@k metric is adopted, common in the literature for temporal action segmentation. A predicted segment is considered a true positive if there exists a ground truth segment of the same class with which the temporal overlap (IoU, Intersection over Union between the $[start, end]$ intervals) exceeds a threshold k; each ground truth segment can be associated with only one true positive. Three variants are calculated, at increasing overlap thresholds $k \in \{0.10, 0.25, 0.50\}$, to evaluate the model's tolerance to small, medium, and large misalignments in phase boundaries:

$$F1@k = \frac{2 \cdot \text{Precision}_k \cdot \text{Recall}_k}{\text{Precision}_k + \text{Recall}_k}, \qquad \text{IoU}(\text{seg}_{pred}, \text{seg}_{gt}) \geq k$$

The three metrics F1@10, F1@25, and F1@50 are finally averaged into a single summary metric, **mean segmental F1**, used as the primary criterion both for early stopping during training (Chapter 4) and for selecting the best candidate (loss function and post-processing strategy) during validation. This choice reflects the project's objective to favor temporally coherent and clinically plausible predictions, rather than solely frame-level accuracy.

4.4 Multi-Label Metrics for Instrument Recognition

The auxiliary instrument task is multi-label: multiple instruments can be present in the same frame. After setting the threshold on the validation set, Precision, Recall, and macro F1 are calculated across the seven instruments, in addition to the micro F1. The **macro F1** assigns the same weight to each instrument and highlights rare classes; the **micro F1** is dominated by the most frequent decisions and describes the overall quality of all binary labels.

The **samples Jaccard** is calculated separately for each frame as the ratio between the intersection and union of the predicted and real instrument sets, then averaged over the frames. The **exact match accuracy** is the fraction of frames in which the entire binary vector of the seven instruments is exact: it is therefore a stricter metric than simple accuracy per single label. In the rare cases where both the predicted and real sets are empty, the implementation applies the convention of the evaluation library used in the notebook.

4.5 Computational Efficiency Metrics

For each run, the number of trainable parameters, the total training time until early stopping, the peak allocated GPU memory, and the inference time on the entire validation and test set are recorded. These indicators do not measure predictive quality, but allow evaluating the trade-off between performance, training cost, and model practicability. The times depend on the hardware, the actual length of the sequences, and the number of executed epochs; they must therefore be interpreted as comparative measures of the specific experimental environment, not as universal hardware benchmarks.

4.6 Evaluation Protocol and Data Leakage Prevention

To ensure the validity of the final evaluation, the experimental protocol strictly follows this sequence: (1) for each combination of feature source and candidate loss, the model is trained and evaluated exclusively on the validation set; (2) the best combination of loss function and post-processing strategy (raw vs. Viterbi) is selected solely based on validation metrics; (3) only at this point is the test set instantiated and used for a single, unrepeatable final evaluation of the selected model and configuration. This precaution, also explicit in the code (*"the test set is instantiated only after candidate/checkpoint/post-processing freeze"*), prevents any form

of implicit optimization on the test set, ensuring that the final metrics reported in Chapter 6 constitute an unbiased estimate of the model's generalization capability.

For the multi-task variant (Chapter 4), the optimal decision threshold for the binary classification of instrument presence is also selected on the validation set, exploring the interval $[0.30, 0.70]$ with a step of 0.05 and choosing the value that maximizes the macro F1; the instrument detection metrics (Precision, Recall, macro/micro F1, sample-level Jaccard, exact match accuracy) are then calculated on the test set with the threshold fixed in this way, following the same principle of separation between selection and final evaluation.

Chapter 5

Results

5.1 Comparison Configuration

The final evaluation includes seven temporal configurations — GRU, BiGRU, MS-CBiGRU, Recognition Transformer, SF-TMN(ASFormer), 10-layer multi-task MS-TCN, and 11-layer multi-task MS-TCN — applied to the three feature sources `original`, `finetuned`, and `teammates`. All main results use the same 32/8/40 split and the selection protocol described in Chapter 4. The metrics reported in Tables 5.1 and 5.2 are expressed as percentages and stem from the single final evaluation on the test set after freezing the checkpoint, loss, and post-processing.

In the tables, `raw` indicates the direct output of the model and `Vit.` the Viterbi decoding selected during validation. Bold highlighting identifies the global maximum of the respective column; it does not imply statistical significance.

5.2 Phase Recognition: Frame-Level Metrics

Feature	Model	Post	Acc.	Prec. M	Rec. M	F1 M	Jacc. M
original	GRU	Vit.	80.44	76.48	83.45	78.31	65.01
original	BiGRU	Vit.	81.10	73.30	86.77	77.16	63.76
original	MS-CBiGRU	Vit.	88.07	81.15	89.11	83.75	72.83
original	Transformer	Vit.	78.24	76.54	70.87	72.21	57.01
original	SF-TMN	raw	91.31	85.89	88.02	86.84	77.18
original	MS-TCN-MT-10L	raw	91.43	86.30	88.09	87.10	77.52
original	MS-TCN-MT-11L	Vit.	91.61	**88.37**	85.79	86.80	77.15
finetuned	GRU	Vit.	85.95	77.61	82.44	79.45	66.48
finetuned	BiGRU	Vit.	87.68	79.07	84.32	81.08	68.90
finetuned	MS-CBiGRU	Vit.	87.29	78.22	84.69	80.74	68.44
finetuned	Transformer	Vit.	86.17	82.70	80.49	80.68	68.02
finetuned	SF-TMN	Vit.	91.62	88.08	87.57	87.73	78.36
finetuned	MS-TCN-MT-10L	Vit.	90.61	84.00	**89.92**	86.37	76.31
finetuned	MS-TCN-MT-11L	Vit.	91.37	87.41	88.67	**87.86**	**78.61**
teammates	GRU	Vit.	83.14	78.34	81.93	79.48	66.58
teammates	BiGRU	Vit.	88.45	79.73	86.71	81.99	70.23
teammates	MS-CBiGRU	Vit.	88.40	79.90	87.66	82.80	71.37
teammates	Transformer	Vit.	77.54	72.03	74.54	71.35	56.36
teammates	SF-TMN	raw	87.31	79.74	78.46	78.89	66.61
teammates	MS-TCN-MT-10L	Vit.	87.02	84.87	83.09	83.62	72.18
teammates	MS-TCN-MT-11L	raw	**92.12**	87.16	87.70	87.34	77.98

Table 5.1: Frame-level metrics on the test set. M = macro; values in percentage.

The highest accuracy, 92.12%, is achieved by the 11-layer multi-task MS-TCN with `teammates` features. The best macro F1 and best macro Jaccard belong instead to the same 11-layer model with `finetuned` features, reaching 87.86% and 78.61% respectively. The best macro recall, 89.92%, is obtained by the 10-layer variant with `finetuned` features. These differences show that there is no single dominant configuration for all frame-level metrics.

5.3 Temporal Quality of Predictions

Feature	Model	Post	Edit	F1@10	F1@25	F1@50	Mean seg. F1
original	GRU	Vit.	65.06	72.99	71.55	62.64	69.06
original	BiGRU	Vit.	79.09	83.62	79.25	67.39	76.76
original	MS-CBiGRU	Vit.	85.71	89.67	87.67	79.67	85.67
original	Transformer	Vit.	44.63	52.49	47.41	38.73	46.21
original	SF-TMN	raw	**86.69**	**91.40**	**91.06**	**82.63**	**88.36**
original	MS-TCN-MT-10L	raw	77.22	79.35	78.17	71.09	76.20
original	MS-TCN-MT-11L	Vit.	81.78	87.17	86.51	78.95	84.21
finetuned	GRU	Vit.	57.21	67.08	64.36	55.94	62.46
finetuned	BiGRU	Vit.	64.36	72.68	70.79	61.10	68.19
finetuned	MS-CBiGRU	Vit.	63.85	72.41	71.06	61.91	68.46
finetuned	Transformer	Vit.	47.87	56.24	51.84	44.07	50.72
finetuned	SF-TMN	Vit.	85.73	89.11	87.79	80.20	85.70
finetuned	MS-TCN-MT-10L	Vit.	85.69	89.44	88.12	82.51	86.69
finetuned	MS-TCN-MT-11L	Vit.	84.26	86.97	86.32	79.15	84.15
teammates	GRU	Vit.	65.70	73.62	72.46	61.74	69.28
teammates	BiGRU	Vit.	79.71	84.98	83.39	73.48	80.62
teammates	MS-CBiGRU	Vit.	82.06	85.85	83.90	74.47	81.41
teammates	Transformer	Vit.	48.10	53.93	50.53	38.22	47.56
teammates	SF-TMN	raw	81.81	85.01	82.04	70.84	79.30
teammates	MS-TCN-MT-10L	Vit.	82.43	87.60	86.61	77.36	83.86
teammates	MS-TCN-MT-11L	raw	81.65	81.90	81.29	75.77	79.65

Table 5.2: Temporal segmentation metrics on the test set; values in percentage.

According to the primary metric, the best configuration is SF-TMN with `original` features: mean segmental F1 88.36%, Edit 86.69%, and the best values at all overlap thresholds. This is followed by the 10-layer multi-task MS-TCN with `finetuned` features (86.69%) and SF-TMN with `finetuned` features (85.70%). The configuration with the highest accuracy, 11-layer MS-TCN with `teammates` features, instead reaches 79.65% mean segmental F1. A high frame-level accuracy thus does not guarantee correct segments free of fragmentation.

The Transformer is the least effective model on temporal metrics: its mean segmental F1 ranges between 46.21% and 50.72%, even though with `finetuned` features it reaches 86.17% accuracy. The 64-second window and the aggregation of overlapping windows seem sufficient to classify many frames, but not to reconstruct the global structure of the intervention with the same quality. Conversely, SF-TMN, MS-TCN, and MS-CBiGRU benefit from processing the entire sequence or from a multi-scale temporal context.

5.4 Effect of Features and Depth

Fine-tuned features do not uniformly improve all models. They provide the best macro F1 and macro Jaccard with the 11-layer MS-TCN and clearly improve the Transformer compared to the other sources, but they reduce the segmental metrics of recurrent models compared to the `original` features. SF-TMN achieves the best temporal result exactly with the `original` features. This indicates an interaction between feature distribution and the inductive bias of the temporal model: the quality of a source cannot be judged with a single architecture.

In the depth comparison, moving from 10 to 11 layers per stage, the mean segmental F1 increases on `original` features from 76.20% to 84.21% (+8.01 points), but decreases on `finetuned` features from 86.69% to 84.15% (−2.54 points) and on `teammates` features from 83.86% to 79.65% (−4.21 points). Accuracy increases in all three cases, especially with `teammates` (+5.10 points), confirming once again the possible misalignment between frame-level quality and segmental quality.

This comparison is not, however, a pure ablation of depth: the validation procedure selected different losses and post-processing for the two depths. To causally isolate the effect of the number of layers, it would be necessary to repeat 10 and 11 layers with identical loss, candidate post-processing, initialization, and training budget, ideally across multiple seeds.

5.5 Auxiliary Task: Instruments

Feature / layer	Threshold	Prec. M	Rec. M	F1 M	F1 micro	Jacc. samples	Exact match
original / 10L	0.30	46.82	**66.23**	**53.60**	70.09	55.14	32.13
original / 11L	0.30	**48.13**	62.02	49.05	69.90	**55.34**	31.49
finetuned / 10L	0.30	24.74	46.78	31.20	64.05	50.54	23.91
finetuned / 11L	0.40	44.75	56.61	48.16	**71.22**	55.12	**34.34**
teammates / 10L	0.30	44.23	62.87	50.02	69.33	54.75	31.12
teammates / 11L	0.30	44.76	64.77	50.16	69.87	55.26	31.67

Table 5.3: Multi-label instrument metrics on the test set; values in percentage. The threshold was selected exclusively on the validation set.

The best macro F1 for instruments is 53.60%, obtained by the 10-layer model with `original` features; the best micro F1 and the best exact match accuracy are instead obtained by the 11-layer model with `finetuned` features. The gap between micro and macro F1 indicates that frequent instruments are recognized better than rare ones. The auxiliary task is therefore informative, but not yet sufficiently balanced for reliable use as an autonomous detector.

5.6 Single-Task MS-TCN Ablation

Loss	Acc.	F1 M	Jacc. M	F1 weighted	Jacc. weighted	Edit
Weighted CE	91.32	**87.81**	**78.49**	91.25	84.12	79.13
Focal $\gamma = 1$	91.43	87.36	77.80	**91.36**	84.34	**82.41**
Boundary-aware	**91.49**	87.00	77.31	91.34	**84.37**	81.15

Table 5.4: Preliminary ablation of the 10-layer single-task MS-TCN with fine-tuned features; values in percentage.

The preliminary experiment shows contained differences in frame-level metrics: boundary-aware loss maximizes accuracy, weighted cross-entropy maximizes macro F1 and Jaccard, while focal loss achieves the best Edit. These values are not included in the main ranking because the export does not contain F1@10/25/50 according to the standardized protocol, and the three variants are already evaluated on the test set. The ablation is therefore reported as an exploratory analysis, not as a bias-free final selection.

5.7 Computational Efficiency

Model	Parameters	Training [min]	Peak VRAM [MB]	Test 40 videos [s]
GRU	1,316,359	2.14–3.64	205–206	2.26–2.32
BiGRU	1,119,751	0.47–0.82	252	0.71–0.75
MS-CBiGRU	1,382,919	0.51–0.92	283	0.68–0.75
Transformer	2,173,959	6.99–16.72	144	0.96–1.04
SF-TMN(ASFormer)	2,259,932	17.25–29.14	336	5.02–8.46
MS-TCN-MT-10L	796,792	0.78–2.10	158–159	0.67–2.18
MS-TCN-MT-11L	862,840	1.19–1.89	166	0.69–2.27

Table 5.5: Computational cost observed on the three feature sources. The intervals reflect differences in the number of epochs until early stopping and in the actual duration of the executions.

Chapter 6

Conclusions and Future Developments

This work uniformly compared recurrent models, Transformer, MS-TCN, and SF-TMN across three ResNet50 feature sources, evaluating them both at the frame level and at the temporal segmentation level. The main result is that **SF-TMN with original features** represents the best configuration for workflow reconstruction: mean segmental F1 of 88.36%, Edit of 86.69%, and best F1@10, F1@25, and F1@50. If the objective were instead to maximize solely frame-level accuracy, the choice would fall on the **11-layer multi-task MS-TCN with teammates features**, which reaches 92.12%.

The difference between these two winners is substantial from an application perspective. In surgical phase recognition, correctly classifying many frames is not enough: a fragmented sequence or one with spurious transitions is less useful for context-aware assistance, video indexing, and workflow analysis. For this reason, the mean segmental F1 remains the selection criterion most consistent with the task, while accuracy, macro F1, and Jaccard should be considered complementary metrics.

Fine-tuned features are not universally superior. They are highly effective with MS-TCN and Transformer, but the original features produce the best results with SF-TMN and MS-CBiGRU. Even the increase from 10 to 11 layers does not determine a constant advantage: it strongly improves accuracy on teammates features, but reduces segmental F1 on finetuned and teammates features. The project thus shows that the feature source, temporal architecture, loss, and post-processing interact with each other and must be jointly selected on the validation set.

The multi-task variant achieves competitive phase performance at a limited computational cost, but instrument recognition remains weaker: the best macro F1 is 53.60%, compared to micro values close to 70%. This gap suggests that the auxiliary task primarily learns frequent instruments. The use of `pos_weight`, focal BCE, balanced sampling, or per-class thresholds therefore constitutes a priority development.

6.1 Study Limitations

The final experiments use a single seed and a single 40-video hold-out; therefore, confidence intervals or a measure of variability due to initialization are not available. Furthermore, the 10-vs-11 layer comparison does not keep the selected loss fixed, and cannot be interpreted as a causal estimate of the depth effect alone. The three feature sources have different lineages and might incorporate preprocessing differences in addition to fine-tuning. Finally, MS-TCN, SF-TMN, BiGRU, and MS-CBiGRU use future context: the results describe an offline system and do not demonstrate equivalent performance in a causal real-time scenario.

The single-task ablation prior to the final protocol is kept separate because it does not contain all standardized segmental metrics and has already used the test set to compare the losses. This

separation avoids presenting as confirmatory an analysis that should be considered exploratory.

6.2 Future Developments

As a first step, the strongest configurations should be repeated over at least 3–5 seeds, reporting the mean, standard deviation, and per-video bootstrap confidence intervals. The comparison between SF-TMN original, MS-TCN-10L finetuned, and MS-TCN-11L finetuned should be accompanied by paired tests on the 40 videos, with correction for multiple comparisons. A true ablation of depth must keep identical the loss, scheduler, seed, and candidate post-processing; if the original design goal was to compare 10 and 12 layers, the 12-layer model must actually be trained, since the current outputs only contain 10 and 11 layers.

From a modeling perspective, a causal variant for online use, a semi-Markov or duration-aware decoder explicitly modeling phase durations, and a lightweight ensemble between SF-TMN and MS-TCN are promising. For the instrument task, positive weights per class, per-instrument thresholds selected on validation, and a multi-label focal loss are recommended. Finally, the analysis of confusion matrices and videos with worse metrics should guide targeted interventions on transitions and rare phases, avoiding optimizing only the global mean.

Bibliography

[1] Florent Lalys and Pierre Jannin. Surgical process modelling: a review. *International journal of computer assisted radiology and surgery*, 9(3):495–511, 2014. doi:10.1007/s11548-013-0940-5.

[2] Nicolas Padoy. Machine and deep learning for workflow recognition during surgery. *Minimally Invasive Therapy & Allied Technologies*, 28(2):82–90, 2019. doi:10.1080/13645706.2019.1584116.

[3] Andru P Twinanda, Sherif Shehata, Didier Mutter, Jacques Marescaux, Michel de Mathelin, and Nicolas Padoy. Endonet: A deep architecture for recognition tasks on laparoscopic videos. *IEEE transactions on medical imaging*, 36(1):86–97, 2017. doi:10.1109/TMI.2016.2593957.

[4] Mitchell Doughty, Karan Singh, and Nilesh R. Ghugre. Surgeonassist-net: Towards context-aware head-mounted display-based augmented reality for surgical guidance. In *Medical Image Computing and Computer Assisted Intervention – MICCAI 2021*, volume 12904, pages 667–677. Springer, 2021. doi:10.1007/978-3-030-87202-1_64.